Push_swap

Documentation & Résumé de Soutenance

abdazero

21 février 2026

Table des matières

1 Objectif du Projet **2**

2 Architecture Générale du Programme **2**

3 Parsing : De `argv` à la Pile **2**
3.1 Objectif 2
3.2 Étapes clés 2
3.3 Gestion des erreurs 2

4 Structure de Données : Liste Chaînée **3**
4.1 Pourquoi une liste chaînée ? 3

5 Les Opérations Implémentées **3**

6 Stratégie de Tri Adaptative **3**
6.1 Cas 2–5 éléments 4
6.2 Cas 6–50 éléments 4
6.3 Cas 50+ éléments : Radix Sort 4
6.3.1 Étape 1 : Indexation 4
6.3.2 Étape 2 : Tri par bits 4
6.3.3 Complexité 4

7 Performances **4**
7.1 100 nombres 4
7.2 500 nombres 5

8 Gestion Mémoire **5**

9 Notions Techniques Maîtrisées **5**

10 Conclusion pour la Correction **5**

11 Résumé Court à Dire à l'Oral **5**

1 Objectif du Projet

Le projet **push_swap** consiste à trier une pile d'entiers en utilisant un ensemble **limité d'opérations**, tout en générant **le minimum d'instructions possible**.

Deux piles sont utilisées :

— **Pile A** : contient les nombres à trier (au départ)

— **Pile B** : pile auxiliaire (vide au départ)

Le but est d'obtenir la pile A triée dans l'ordre croissant.

2 Architecture Générale du Programme

Le programme se décompose en 3 grandes parties :

1. **Parsing & Validation** des arguments
2. **Structure de données** (liste chaînée)
3. **Algorithmes de tri** adaptés à la taille

3 Parsing : De `argv` à la Pile

3.1 Objectif

Transformer les arguments texte en une pile valide et sécurisée.

3.2 Étapes clés

1. **Vérification du nombre d'arguments**
   — Si `argc < 2` → rien à afficher, le programme se termine.
2. **Découpage des arguments**
   — Utilisation de `ft_split` pour gérer les cas :
   `./push_swap "42 21 84"`
3. **Validation stricte**
   — Vérification du format numérique (`+`, `-`, chiffres)
   — Conversion via `ft_atol` pour détecter les overflow
   — Vérification que le nombre est dans l'intervalle `INT_MIN / INT_MAX`
   — Vérification des doublons
4. **Construction de la pile**
   — Les nombres sont ajoutés **à la fin** pour respecter l'ordre donné.

3.3 Gestion des erreurs

À la moindre erreur :

— Libération complète de la mémoire

— Affichage de `"Error"` sur `stderr`

Parsing robuste = point très important en correction.

4 Structure de Données : Liste Chaînée

J'ai choisi une **liste chaînée simple** :

```
typedef struct s_node
{
    int             value;
    int             index;
    struct s_node   *next;
} t_node;
```

4.1 Pourquoi une liste chaînée ?

- **Avantages**
  - Opérations `push`, `pop`, `rotate` en **O(1)**
  - Taille dynamique
  - Idéal pour structure LIFO (pile)
- **Pourquoi pas un tableau ?**
  - `push`/`pop` coûteraient O(n) (décalage mémoire)
  - Moins adapté à la logique du projet

Choix justifié techniquement et cohérent avec les contraintes.

5 Les Opérations Implémentées

Type	Instructions
Swap	sa, sb, ss
Push	pa, pb
Rotate	ra, rb, rr
Reverse rotate	rra, rrb, rrr

Chaque opération :
- Modifie uniquement les pointeurs
- Affiche l'instruction correspondante
- Respecte strictement le sujet

6 Stratégie de Tri Adaptative

Le programme adapte l'algorithme selon la taille :

```
if (size <= 5)
    algorithmes optimises
else if (size <= 50)
    tri par insertion dans B
else
    radix sort
```

6.1 Cas 2–5 éléments

Algorithmes codés à la main pour garantir un nombre **minimal d'opérations**.
Exemple :
— 3 nombres → gestion des 6 permutations possibles.
Optimisation maximale pour petits cas.

6.2 Cas 6–50 éléments

Tri par insertion dans B :
— On envoie progressivement les plus petits vers B
— B reste triée
— On remonte tout vers A
Bon compromis entre complexité et nombre d'instructions.

6.3 Cas 50+ éléments : Radix Sort

6.3.1 Étape 1 : Indexation

On transforme les valeurs en positions triées.
Exemple :

```
Valeurs : [42, 21, 84]
Index   : [1, 0, 2]
```

Cela permet de travailler sur des nombres de 0 à n-1.

6.3.2 Étape 2 : Tri par bits

Pour chaque bit :

```
if ((index >> i) & 1)
    ra
else
    pb
```

Puis on remonte tout avec `pa`.

6.3.3 Complexité

Radix sort → **O(n log n)**
Très adapté aux contraintes push_swap.

7 Performances

7.1 100 nombres

— Résultat : **1084 instructions**
— Barème : < 1100 → 3 points

7.2 500 nombres

— Résultat : **6784 instructions**
— Barème : $< 7000 \rightarrow 4$ points
Résultats stables et constants
Algorithme non sensible à l'ordre initial

8 Gestion Mémoire

— Chaque `malloc` a son `free`
— Fonction dédiée pour libérer la pile
— Libération en cas d'erreur
— Tests Valgrind → aucune fuite
Point indispensable pour valider le projet.

9 Notions Techniques Maîtrisées

— Pointeurs et double pointeurs
— Manipulation de listes chaînées
— Opérateurs bit à bit (`»`, `&`)
— Complexité algorithmique
— Gestion des erreurs propre
— Organisation modulaire du code

10 Conclusion pour la Correction

Mon projet push_swap :
Parsing robuste (erreurs, overflow, doublons)
Gestion mémoire parfaite
Structure adaptée (liste chaînée)
Algorithmes optimisés selon la taille
Radix sort performant
Résultats stables (1084 / 6784)

11 Résumé Court à Dire à l'Oral

J'ai structuré mon projet en trois parties : parsing sécurisé, gestion via liste chaînée pour garantir des opérations en O(1), et stratégie de tri adaptative.

Pour les petits cas, j'utilise des algorithmes optimisés. Pour les grandes tailles, j'utilise un radix sort sur index, ce qui me donne une complexité proche de O(n log n).

Mon parsing gère tous les cas d'erreur, et ma gestion mémoire est propre sans fuite.

Mes performances sont stables : 1084 coups pour 100 nombres et 6784 pour 500.